

# Welcome to CMSC 104

## Section 02 Fall 2026

Alfred W. Arsenault  
24 August 2026

# Administrative notes

- Each lecture will have a set of slides that will guide the course content that day
- Slides are posted - usually ahead of time - on the class GitHub repo
- The second slide of each day's notes will be an “administrative notes” slide
  - I will remind you of upcoming assignments
  - When the quizzes will be
  - When classwork is due
  - Etc.
- Note: this is provided as a “quick reference” for you to look up due dates/test dates. Every now and then I make a mistake, which I correct once it's identified.
  - If you have a question, ASK.

About the instructor

# Alfred W. Arsenault, Adjunct Professor

Office: ITE 353

Office hours: before class, typically 4:30 - 5:30, and after class. Other times by appointment

E-mail: [arsenaul@umbc.edu](mailto:arsenaul@umbc.edu).

Phone (cell): 410-746-4731

***Please note that I am an adjunct professor. This is not my full-time job. I work full-time for the MITRE Corp in Annapolis Junction. I also teach graduate school at UM-Baltimore. When I am here this job has my undivided attention. But I may not be available for “walk-in” office hours.***

# More about me

Education: BS Physics and BS Computer Science, Southeastern Louisiana University. MS Computer Science, Purdue University. MS Data Science, University of California-Berkeley.

Computer Science interests:

- Cyber security
- Networking
- Data analytics

Non-Computer Science interests:

- Competitive ballroom dancing (with my wife)
- Youth sports - formerly ran HCYP Girls' Fastpitch softball program; coached softball, baseball, soccer, basketball and volleyball
- In the olden days: I had a private pilot's license (not current), and jumped out of airplanes a bunch of times

# And one more about me

Married, four kids, one dog

- Kids:
  - Beth: BS, English/Journalism, Hood College
  - Will: BS, Computer Science, UMBC; MS, Computer Science, Johns Hopkins
  - Kate: BS, Applied Math, UMCP; MS, Technology Leadership, George Washington
  - Mary: BS, Media & Communications Studies; double major in Psychology, UMBC
    - She actually works downstairs in this building - videography/livestreaming sports events
- Dog:
  - Doug, Beagle mix, 12 years old; mostly just lies around looking cute.

Why are we here?  
*An overview of the class*

# Computer Science

- Yes, we're going to teach you how to program a computer
  - Specifically, using the Python programming language
- You're going to learn other things about computers, too
  - What's the difference between hardware and software
  - What's the difference between system software and application software
  - High-level vs. machine vs. assembly language
  - Syntax and semantics - especially, syntax errors and semantic errors
- The point is not to teach you to be a Python coder
  - You can learn that in a week-long bootcamp
  - And that's not what most of you want to be, anyway.
- The point is to get you to understand how computers work, and why

## ***But I've never coded before!!!***

- No problem - this class is specifically designed for people with no experience
  - Remember this is not your major!
- And congratulations - you will write your first computer program before the end of this class
- What if I've programmed in another language but don't know Python?
  - You shouldn't have a problem - Python is relative easy to learn
- Hey, I've been programming in Python since I was 7 and just signed up for this because I need an easy A
  - Cool - just don't interfere with your classmates. And you just might learn something anyway

# Syllabus Review

And now let's program!

# First some context you'll need

- We'll get deeper into this on Wednesday, but for now this will get you started
- There are a few things you need to actually get a program written, debugged, and running:
  - An editor, in which you can write and edit your program
  - An interpreter or compiler, to translate your program into a language the computer understands and can execute
  - A computer that can execute or “run” the program
  - A place to see the output or results of your program
- When you have all of these together in a single tool, that's called an “Integrated Development Environment” or “IDE”

# There are lots of choices for free IDEs:

- VSCode, from Microsoft
- PyCharm, from JetBrains
- IDLE, made available as part of Python
- The book uses IDLE, and that's okay for you to use. Although I will often use PyCharm, just because it's simpler for me. It's entirely your choice; use what you can learn.

# Let's write this program

Go to <https://www.python.org/download>

Download the version of python appropriate for your computer

Stuff you're supposed to learn this week  
*(don't worry; we'll go through it again on Wednesday)*

Class GitHub Repo:

[https://github.com/Al-Arsenault201/104\\_Spring\\_2026](https://github.com/Al-Arsenault201/104_Spring_2026)

# An overview of computer systems

## Hardware and Software

**Hardware:** those physical parts you can reach out and touch

**Software:** the abstract logic that you can't see or touch; it executes on the hardware

## Software Hierarchy

Operating Systems

Applications

## Networks and Networking

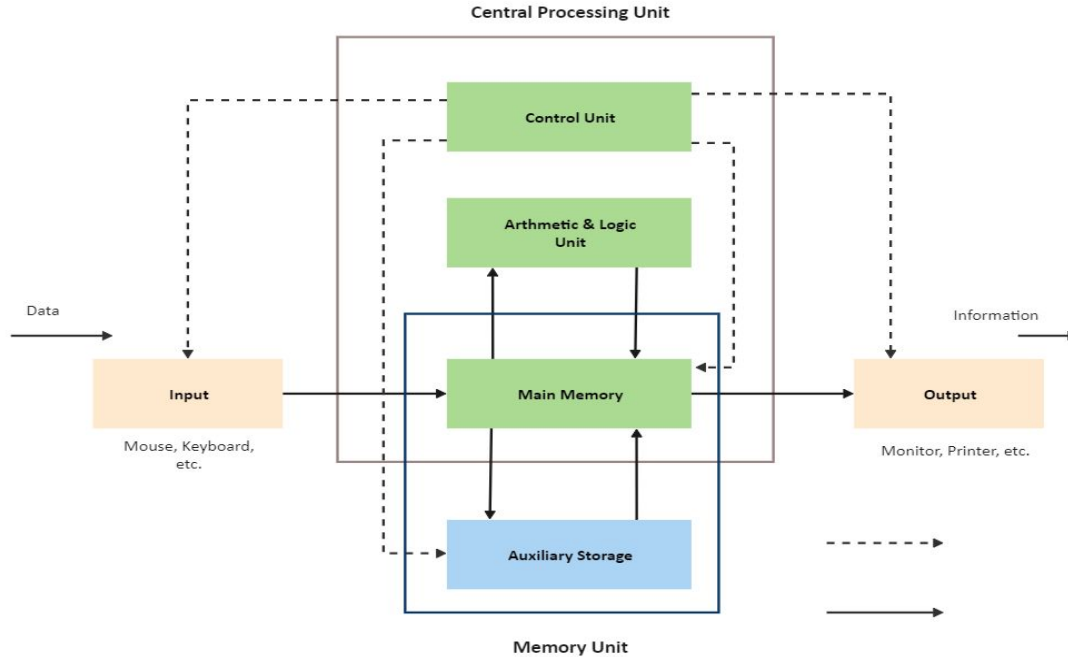
The Internet

The World-Wide Web

Web Services

## Cloud computing

# Typical computer architecture



From <https://edraw.wondershare.com/article/computer-architecture.html>

Phone, tablet, laptop,  
desktop - it really doesn't  
matter; the ideas are the  
same

# Networks, Clouds and Virtual Machines

A network is a bunch of computers connected together to improve productivity

- Exchange information
- Work together to solve problems
- An “internet” was a network of networks; now there’s just “the Internet”

A “cloud” (see next slide)

# Clouds

"a paradigm for enabling network access to a scalable and elastic pool of shareable physical or virtual resources with self-service provisioning and administration on-demand,"

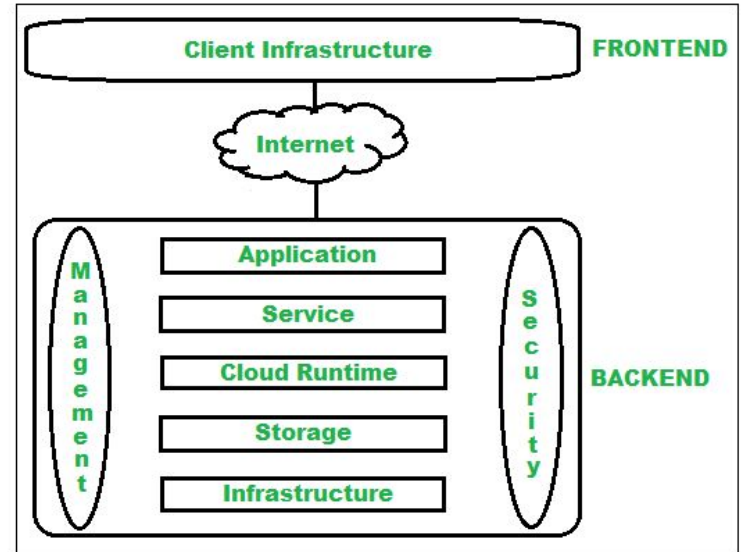
In plain English:

- A service for computing provided by somebody (Amazon, Google, Microsoft, Oracle,...) or even your own organization
- On-demand self-service - you get resources when you need them
- Resource pooling - provider has everything; any customer can get needed resources
- Broad network access - you don't have to be sitting in a room or on a campus
- Rapid elasticity - you need more compute power? You got it (just pay for it)
- Measured service - pay for what you use

Why do you care?

- Because so much of computing today and going forward is based on clouds

From <https://www.geeksforgeeks.org/cloud-computing/architecture-of-cloud-computing/>



# Virtual Machine

A “pretend” hardware/software computer constructed entirely out of software on a bigger computer

- Software emulates (pretends to be) the actual hardware, so to your program there's no difference

Popular because computing is so cheap that most computers today are way more powerful than would be needed.

“Container” - related but different

- Figure out what each application really needs; package that together
- Less than a full VM
- Think of shipping containers on ships & trains

# Where do the software and hardware fit in these pictures?

The hardware is what you can see and touch

- Display screen/monitor
- Keyboard
- Mouse
- Touch pad
- Fingerprint reader/faceID reader

There is ***system software*** that tells every piece of hardware what to do

There is ***application software*** that performs “useful” operations:

# System software vs. Application software

**System software** is the software that controls the hardware device

- **The Operating System** is the overall control program
  - Windows, MacOS, Apple iOS, Android, Linux, ...
- **Device drivers** control the interactions of the control unit with the keyboard, mouse, screen, USB drive, hard drive or SSD, etc.

**Application software** does “useful things”

- “Productivity apps” like Google Docs/MS Word
- Grades homework & tests
- Runs video games
- Manages money/does taxes
- Shows videos/live events

# A quick digression on operating systems

1960s/1970s: project at MIT to build MULTICS - a multi-user, high performance, secure operating system

1970s: AT&T Bell Laboratories pulls out of MULTICS; Thompson, Ritchie and Kernighan write UNIX

1970s/1980s: Bell Labs shares UNIX code; UC-Berkeley writes its own “improved” version & distributes it

- lots of lawsuits follow about who owns what code

1990s: Linus Torvalds, a student in Finland, writes a version of Unix he calls “Linux” and distributes it free - “open source software”

- Decades of advancements follow, but Linux still embodies many of the original UNIX principles

# Today:

Windows: Microsoft-proprietary, but has incorporated some principles from other operating systems

Apple MacOS and iOS: derived from the version of UNIX that was written at UC-Berkeley

Android: derived from Linux; purchased and controlled by Google

Linux: still open source and free; version control by Linus Torvalds and a small team.

Many versions:

- RedHat
- Kali
- Debian
- Ubuntu

# Data Hierarchy

**Bit** - one binary digit - a “1” or a “0”

**Character** - a sequence of bits (often, 8 or 16) that together represent one character in the language you care about - e.g, an “A” or a “9”

**Fields** - a collection of characters that together represent something meaningful - e.g., a “student last name” or a “student ID” or a “test grade”

**Records** - a collection of fields related to the same entity - e.g., everything UMBC knows about one student that’s relevant to this class

**Files** - a collection of records treated together - e.g., the student records of all of you in this class

**Database** - one or more files that contain all data relevant to some project/effort

**Big Data** - huge collections of data related in some way - e.g, “all Tweets sent out on Monday,” “all WhatsApp messages sent from Germany

# Programming

# Language Hierarchy

## Machine languages

- What actually runs on the hardware
- A string of 1's and 0's (different voltages, phases, whatever)
- Unique to every hardware manufacturer
- You're never going to have to deal with this (be grateful!)

## Assembly languages

- Quasi-human readable instruction sets
- One instruction per line of code
- Has to be assembled (translated to machine code) prior to executing

## High-level languages

- What we deal with in this class
- Somewhat human-readable
- Examples:
  - Python (this class)
  - C/C++ (what this class used to cover)
  - Java
  - JavaScript

# Where does Programming fit?

Aka “why do I need to know how to program?”

Everywhere

- What kind of programs do you want to write?
  - Operating systems? You can contribute to Linux distros if you want
  - Networking
  - Games
  - Financial applications
- The language you use and the operating system you write for are tied to the kind of programming you want to do
  - C/C++ - operating systems, compilers, applications that need high performance
  - Python - big data, analytics, graphics, and even simple programs to perform a task
  - JavaScript - web services

# The Python Programming Language

# Python history

Released in 1991 by Guido van Rossum, a Dutch programmer

- Controlled by van Rossum and a 5-member steering committee that decides what new features to add
- There's a new core version every few months

<https://www.python.org> is the location for the most current (and previous) “official versions”

# Python philosophy

Extensibility is key

- Supports defining and using modules to provide new capabilities

Additionally:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Readability counts.

# Syntax and semantics

Every language has its syntax and semantics

- Syntax is the set of rules for constructing expressions in the language:
  - “Every sentence must have a subject and a predicate”
  - “Every paragraph must have a topic, and be related to that topic.”
- Semantics is understanding what expressions mean:
  - Often related to the context in which you’re speaking:
    - “Move those axes to the right”
      - Are we in the woods, and I want you to pick up some tools and carry them?
      - Are we working on a computer screen, and I want you to click on a line and drag it?

# Python syntax and semantics

We'll be learning a lot about these this semester

- What to type if you want your program to display something on the screen?
- What's the difference between (, [, and {?
- Those are ***syntax*** issues
  - If you break a Python syntax rule, your program doesn't run
- What happens when you type

```
print(5+4**3)
```

- ***Semantics*** means figuring out what will happen when a statement is executed
- When you get the semantics wrong, your program runs but doesn't give you the answer you wanted or expected.

# Here's a real quick (ungraded) assignment I want everyone to do

Write your first Python program - print out the phrase "hello, world"

- Due to the weather, I made this part of Classwork 1

# So what do we expect you to know from this lecture?

What's hardware, and what's software?

What's system software, and what's application software?

What's an operating system?

What language is taught in this class? Why?